

```

import numpy as np
import matplotlib.pyplot as plt

# List of files and titles
files = [
    ("HposHneg.txt", r"$H^+H^- $"),
    ("A0HPos.txt", r"$A^0 H^+$"),
    ("A0HNeg.txt", r"$A^0 H^- $"),
    ("H0HPos.txt", r"$H^0 H^+$"),
    ("H0HNeg.txt", r"$H^0 H^- $"),
    ("h+tow+A0.txt", r"$h \to W^+ A^0$"),
    ("h-tow-A0.txt", r"$h \to W^- A^0$"),
    ("h+tow+h0.txt", r"$h \to W^+ h^0$"),
    ("h-tow-h0.txt", r"$h \to W^+ h^0$")
]

plt.figure(figsize=(18, 12))

for i, (file, title) in enumerate(files, 1):
    data = np.loadtxt(file, skiprows=1, dtype=str)

    x = data[:, 1].astype(float)
    sigma_pb = data[:, 2].astype(float)

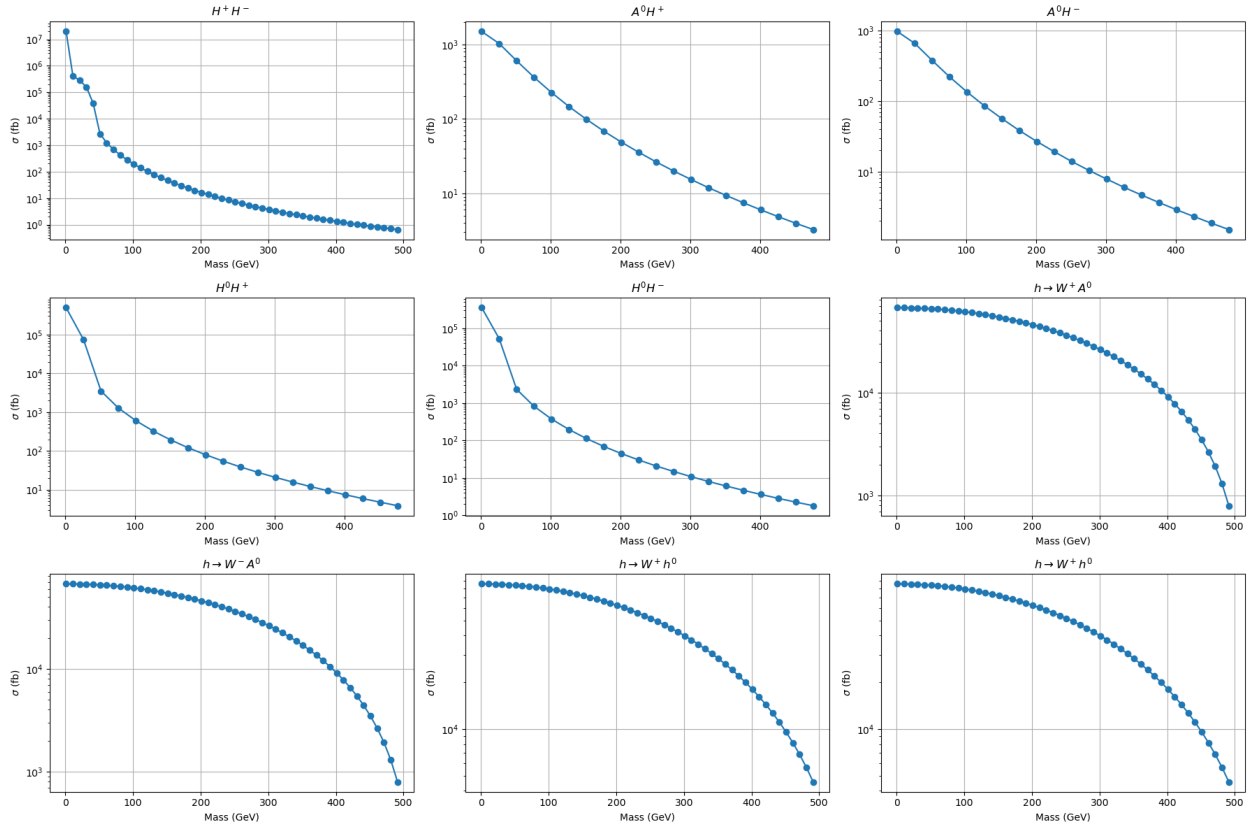
    # Convert pb → fb
    sigma_fb = sigma_pb * 1e3

    plt.subplot(3, 3, i)
    plt.plot(x, sigma_fb, marker='o', linewidth=1.5)

    plt.yscale('log')
    plt.xlabel("Mass (GeV)")
    plt.ylabel(r"$\sigma$ (fb)")
    plt.title(title)
    plt.grid(True)

plt.tight_layout()
plt.show()

```



```
import uproot
import awkward as ak
import pandas as pd
import os
import glob

# --- 1. Physics Parameters ---
cross_section_fb = 500.0 # Total theoretical signal
luminosity_fb_inv = 137.0 # LHC Run 2 Integrated
Luminosity[cite: 2]
total_expected_yield = cross_section_fb * luminosity_fb_inv # 68,500
events[cite: 2]

# --- 2. Path Configuration from your Image ---
base_path = r"C:\Users\Raghav_Y\Desktop\Projects\Sem 6 project\
IDM_Project"

# Folder names exactly as seen in your file explorer
folders = {
    "IDM_Hadronic": "IDM_Hadronic",
    "IDM_SemiLeptonic": "IDM_SemiLeptonic",
    "IDM_Dileptonic": "IDM_Dileptonic"
}

results = []
```

```

print("Starting efficiency analysis...\n")

# --- 3. Efficiency Calculation Loop ---
for channel, folder_name in folders.items():
    # Construct search path for the root file
    search_path = os.path.join(base_path, folder_name, "**", "*.root")
    root_files = glob.glob(search_path, recursive=True)

    if not root_files:
        print(f"Skipping {channel}: No .root file found in {folder_name}")
        continue

    file_path = root_files[0]

    with uproot.open(file_path + ":Delphes") as tree:
        # Load Lepton Multiplicity
        electrons = tree["Electron.PT"].array()
        muons = tree["Muon.PT"].array()

        total_in_file = len(electrons)
        n_leptons = ak.num(electrons, axis=1) + ak.num(muons, axis=1)

        # Determine "Passed" events based on channel reconstruction
        if channel == "IDM_Hadronic":
            passed = ak.sum(n_leptons == 0)
        elif channel == "IDM_SemiLeptonic":
            passed = ak.sum(n_leptons == 1)
        elif channel == "IDM_Dileptonic":
            passed = ak.sum(n_leptons == 2)

        # Calculate Selection Efficiency (epsilon)[cite: 1]
        efficiency = passed / total_in_file

        # Calculate ACTUAL Signal (S) for the report
        # Formula: Total Possible Events * Efficiency
        actual_s = total_expected_yield * efficiency

        results.append({
            "Channel": channel,
            "File Events": total_in_file,
            "Passed": passed,
            "Efficiency (%)": f"{efficiency*100:.2f}%",
            "Actual Signal (S)": f"{actual_s:.1f}"
        })

# --- 4. Final Output Table ---
df = pd.DataFrame(results)

```

```

print("="*75)
print(f"{'Channel':<20} | {'Efficiency':<12} | {'Actual Signal (S) at  
137 fb-1'}")
print("-" * 75)
for index, row in df.iterrows():
    print(f"{'row['Channel']':<20} | {'row['Efficiency (%)']':<12} |  
{row['Actual Signal (S)']}" )
print("="*75)

```

Starting efficiency analysis...

```

=====
=====
Channel          | Efficiency    | Actual Signal (S) at 137 fb-1
-----
-----
IDM_Hadronic     | 99.94%       | 68458.9
IDM_SemiLeptonic | 63.17%       | 43271.5
IDM_Dileptonic   | 44.36%       | 30386.6
=====
=====

```